# Deployment & Production Engineering

### 1. The POC-to-production gap

Most AI pilots never reach production, and the cause is rarely the model. It's **reliability, integration, cost control, monitoring, security review and ownership**. A demo is the happy path; production is the happy path plus outages, rate limits, malformed inputs, adversarial users, cost spikes, deprecations, and the question of who gets paged. The deliverable that closes it is a **production-readiness review**: a red/amber/green checklist plus a remediation backlog.

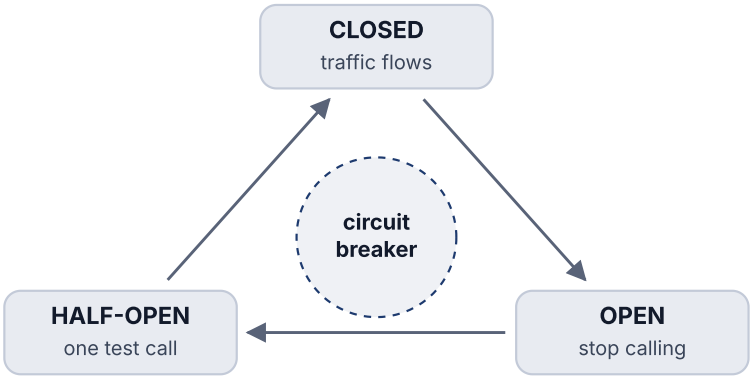
### 2. Serving: API vs self-host

- **Continuous batching** (adding and removing requests token by token to keep GPUs busy) is why API prices collapsed; **speculative decoding** (a small draft model proposes, the big one verifies) gives 1.5 to 3× faster decode
- Self-hosting cost = GPUs + engineering + **utilisation**. A single tenant's spiky traffic leaves GPUs mostly idle, so **APIs win on cost** unless volume is high and steady
- Real reasons to self-host: **data residency**, deep customisation, latency locality
- Middle path: hyperscaler **private endpoints** (Bedrock, Vertex AI, Azure AI Foundry) with in-region processing and a no-training data agreement

### 3. Reliability primitives

- **Timeouts** on every call · **Retries with exponential backoff + jitter** on transient errors (429, 5xx) · **Idempotency keys** so a retry doesn't double an action
- **Fallback chain**: primary model → secondary provider → honest degraded answer
- **Circuit breaker**: after N failures stop calling the failing provider (open), test with one request after a cool-down (half-open), resume on success (closed)
- **Bulkheads** (separate connection pools per dependency), **backpressure** (shed load when the queue is full), a **token-bucket** rate limiter, **graceful degradation** (serve cached or simpler answers)

✎ DIAGRAM




N failures → open; after a cool-down → half-open; a successful test → closed.

### 4. Shipping AI like software

1. Prompts, examples, tool schemas and **model-version pins** in git
2. **CI gate**: the eval suite runs on every change; regressions block the merge
3. Build a container, tag it with the commit
4. Staged rollout: staging → canary
5. **Instant rollback** to the pinned previous version

- Secrets in a secrets manager, never in code; autoscale on queue depth and concurrent requests, not just CPU

### 5. Release strategies

✎ DIAGRAM

| Strategy | How | Use for |
|---|---|---|
| **Canary** | 1 → 5 → 25 → 100% of traffic | Default for prompt and model changes |
| **Blue-green** | Full new copy, flip all at once | Clean cutovers with instant revert |
| **Shadow** | Copy traffic, don't show output | Testing a model swap on real traffic |
| **A/B** | Randomised cohorts, compare outcomes | Deciding which version is better |

Canary bounds the blast radius; shadow lets you test on real traffic with zero user risk.

### 6. Observability and cost

- **Traces** (one ID stitching retrieve → rerank → generate → tool), **metrics** (p50/p95/p99 latency, error rate, tokens, cost per request, cache-hit rate), **logs** (prompt, completion, model version, outcome)
- Watch **p95**, not the average
- **LLM FinOps**: spend by feature and customer, budgets and alerts, max-token caps, a cache-hit-rate dashboard
- **Online evals** on a sample of live traffic; watch for **drift**

### 7. Streaming, APIs and protocols

- **Streaming** sends tokens as they're generated: it doesn't make generation faster, it makes the first words appear in about a second
- **SSE** (server-sent events): one-way, server → browser, plain HTTP, auto-reconnect. The standard for streaming LLM answers
- **WebSockets**: two-way persistent connection. For real-time voice and live collaboration
- Long jobs: **polling**, **webhooks**, or the **batch API** (~50% cheaper)
- Rate limits: **RPM** (requests/min) and **TPM** (tokens/min); a **429** means back off with jitter and queue

✎ DIAGRAM




One-way is SSE, two-way is WebSockets.

### 8. Incidents and on-call

- AI incident types: provider outage, silent quality regression, cost spike, safety incident, model deprecation deadline
- **SLOs** (p95 latency, availability, minimum quality score) with alerts and a **runbook** per alert

- **Blameless post-mortem**, and the fix that matters: add the failure to the eval set
- Model deprecation playbook: eval candidates → shadow-test → canary → re-tune → update pins

SAY IT

*"A demo is the happy path. Production adds timeouts, retries with backoff, a fallback provider, a circuit breaker, eval-gated releases with canaries, and per-request cost and quality tracking."*